

N5_Introduccio_a_la_visualitzacio_de_dades

November 26, 2025

1 Tutorial de Matplotlib

Matplotlib és una llibreria específica per a la representació de dades en Python, ja que permet crear una àmplia varietat de gràfics amb certa capacitat de personalització.

Tot i que existeixen llibreries més modernes i simplificades, tenir coneixements de Matplotlib segueix sent aconsellable per a qualsevol persona que vulgui realitzar representacions gràfiques de dades.

Aquest tutorial pretén ser una introducció de Matplotlib en el que es mostrarà com crear diferents tipus de gràfics i com personalitzar-los a través d'exemples simples.

1.0.1 Abans de començar

Abans de començar els detalls de com crear visualitzacions amb Matplotlib és necessari que coneguis algunes aspectes bàsics d'aquesta llibreria.

Importació de la llibreria Com ja hem comentat amb anterioritat, abans de poder utilitzar una llibreria especialitzada de Python l'haurem d'importar. Igual que en el cas de Numpy o Pandas, per a la importació de la llibreria Matplotlib també utilitzarem una abreviatura (“alias”) que acabat esdevenint un estàndard.

```
[ ]: import matplotlib.pyplot as plt
```

Configuració d'estils El mètode `plt.style` per definir un estil estètic per a totes les gràfiques realitzades a partir de la seva configuració.

```
[ ]: plt.style.use('classic')
```

L'estil ‘classic’ estableix l'estil clàssic de la llibreria Matplotlib. Però n'existeixen d'altres com: ‘seaborn’, ‘ggplot’ o ‘bmh’ i molts més que podràs consultar executant:

```
[ ]: plt.style.available
```

```
[ ]: ['Solarize_Light2',  
      '_classic_test_patch',  
      '_mpl-gallery',  
      '_mpl-gallery-nogrid',  
      'bmh',  
      'classic',  
      'dark_background',
```

```
'fast',
'fivethirtyeight',
'ggplot',
'grayscale',
'petroff10',
'seaborn-v0_8',
'seaborn-v0_8-bright',
'seaborn-v0_8-colorblind',
'seaborn-v0_8-dark',
'seaborn-v0_8-dark-palette',
'seaborn-v0_8-darkgrid',
'seaborn-v0_8-deep',
'seaborn-v0_8-muted',
'seaborn-v0_8-notebook',
'seaborn-v0_8-paper',
'seaborn-v0_8-pastel',
'seaborn-v0_8-poster',
'seaborn-v0_8-talk',
'seaborn-v0_8-ticks',
'seaborn-v0_8-white',
'seaborn-v0_8-whitegrid',
'tableau-colorblind10']
```

La configuració d'estils està disponible a partir de la versió 1.5 de Matplotlib. Si utilitzes una versió més antiga, només tindràs accés a l'especte determinat.

Com i quan es mostren els gràfics resultants Quan utilitzem el mètode `plt.show()` dins d'un *script* i l'executen s'inicia una cerca de tots els elements gràfics configurats i s'obriran una o més finestres que ens mostraran les teves figures resultants.

Per defecte, Jupyter Notebook mostra els gràfics generats amb Matplotlib directament dins de les cel·les del notebook i no obre cap finestra. Però si estàs utilitzant una versió antiga de Jupyter Notebook, o si vols assegurar-te que els gràfics es mostrin correctament dins del notebook, hauries d'executar la comanda `%matplotlib inline` al començament del teu document per tal que els gràfics es mostrin automàticament dins de les cel·les.

Guardar les gràfiques generades El mètode `plt.savefig()` de Matplotlib permet guardar les figures generades en una gran varietat de formats: PNG, JPG, SVG, PDF, entre d'altres.

El format del fitxer resultant dependrà de l'extensió del nom de fitxer que indicarem com a paràmetre d'entrada. Aquí un exemple:

```
plt.savefig('grafic_habitants.jpg')
```

La llista de formats suportats depen del sistema operatiu però els podeu consultar de la següent manera:

```
[ ]: plt.gcf().canvas.get_supported_filetypes()
```

```
[ ]: {'eps': 'Encapsulated Postscript',
      'jpg': 'Joint Photographic Experts Group',
      'jpeg': 'Joint Photographic Experts Group',
      'pdf': 'Portable Document Format',
      'pgf': 'PGF code for LaTeX',
      'png': 'Portable Network Graphics',
      'ps': 'Postscript',
      'raw': 'Raw RGBA bitmap',
      'rgba': 'Raw RGBA bitmap',
      'svg': 'Scalable Vector Graphics',
      'svgz': 'Scalable Vector Graphics',
      'tif': 'Tagged Image File Format',
      'tiff': 'Tagged Image File Format',
      'webp': 'WebP Image Format'}
```

<Figure size 640x480 with 0 Axes>

És recomanable guardar la imatge abans de cridar a `plt.show()` perquè aquest mètode mostra el gràfic per pantalla i, en alguns entorns, pot buidar la figura actual fent que la imatge es guardi totalment en blanc.

1.0.2 Creació d'un DataFrame

Per tal de il·lustrar els diferents exemples de gràfics simples amb Matplotlib creem un DataFrame d'exemple:

```
[ ]: import pandas as pd

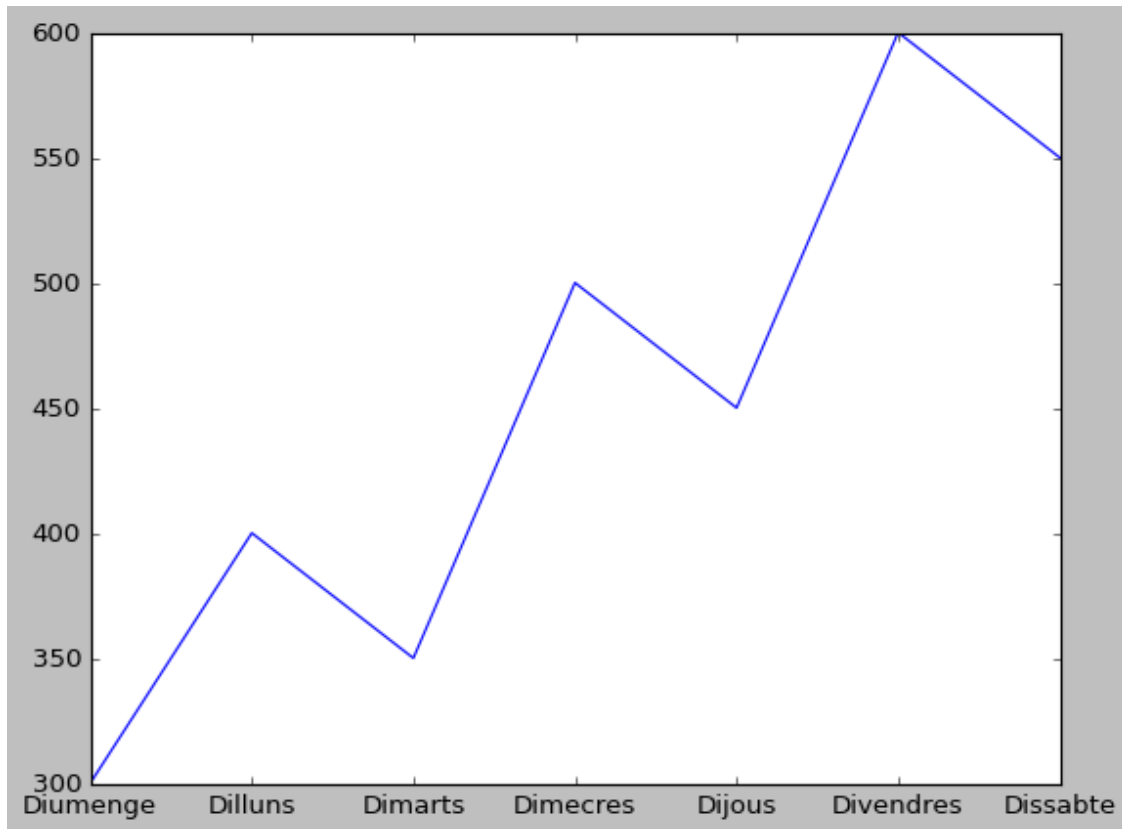
data = {
    'Dia': ['Diumenge', 'Dilluns', 'Dimarts', 'Dimecres', 'Dijous',
    ↪ 'Divendres', 'Dissabte'],
    'Visites': [300, 400, 350, 500, 450, 600, 550],
    'Vendes': [20, 30, 25, 35, 34, 50, 47]
}
df = pd.DataFrame(data)
df
```

```
[ ]:      Dia  Visites  Vendes
0  Diumenge      300      20
1   Dilluns      400      30
2   Dimarts      350      25
3  Dimecres      500      35
4   Dijous      450      34
5  Divendres      600      50
6  Dissabte      550      47
```

1.0.3 Creació d'un gràfic lineal bàsic

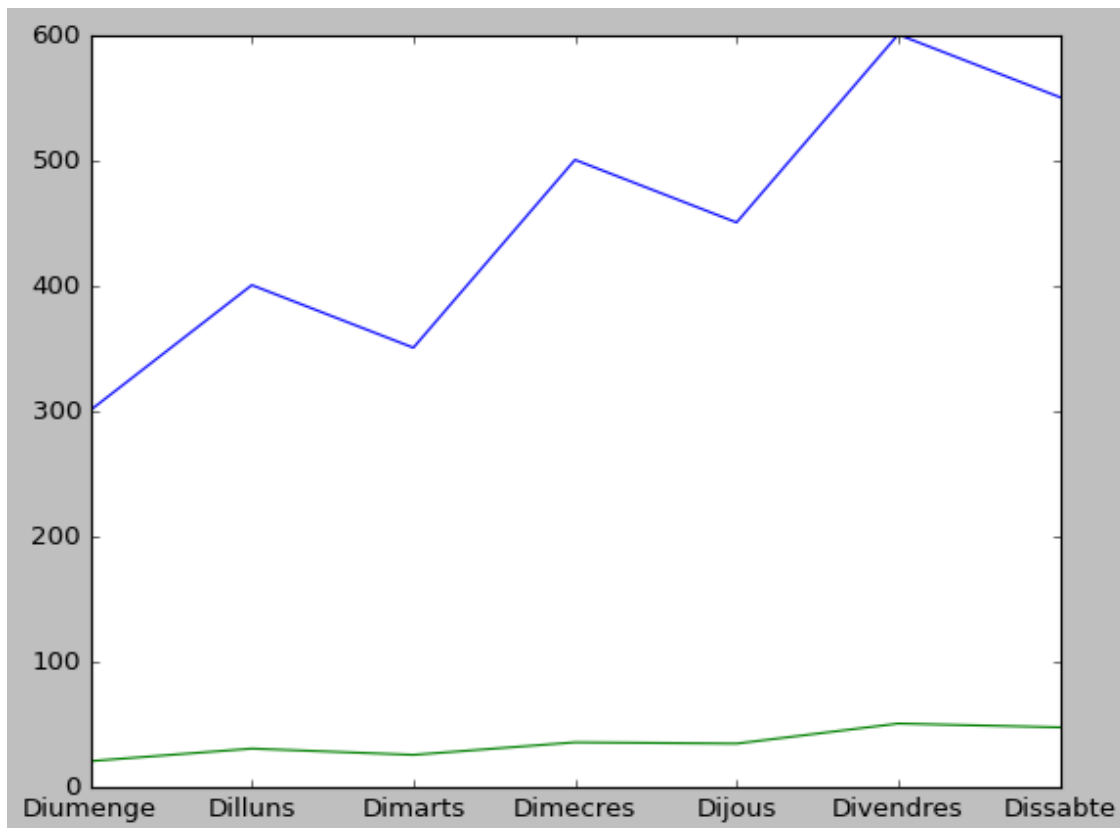
Utilitzarem les dades generades per representar-les gràficament i així obtenir un gràfic lineal bàsic.

```
[ ]: plt.plot(data['Dia'],data['Visites'])  
plt.show()
```



Podem anar pintant tants gràfics com vulguem i es mostraran en el moment que cridem el mètode `plt.show()`.

```
[ ]: plt.plot(data['Dia'],data['Visites'])  
plt.plot(data['Dia'],data['Vendes'])  
plt.show()
```



Segurament et semblaran gràfics molt simples però, com ja hem comentat anteriorment, Matplotlib ens permet personalitzar certs aspectes. Veurem els més bàsics en els següents apartats.

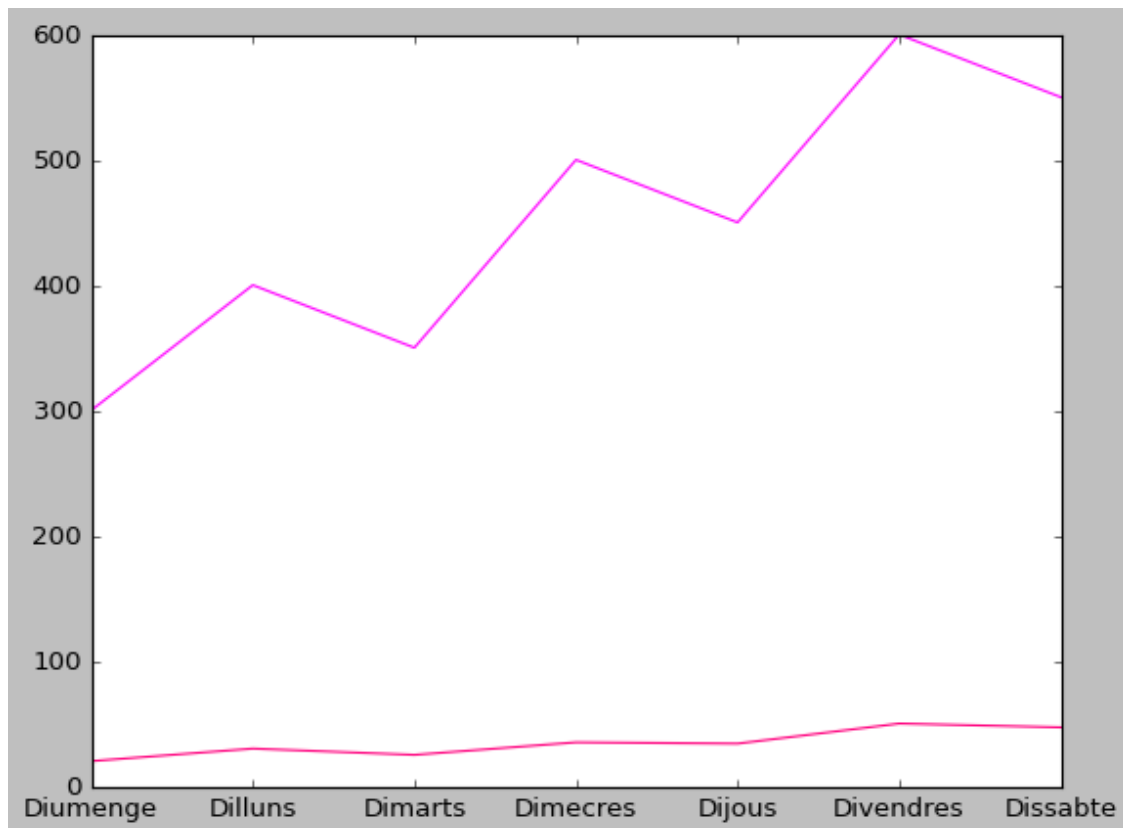
1.0.4 Personalització dels gràfics

Colors de la línia Matplotlib permet utilitzar diversos colors per a les línies dels gràfics mitjançant l'argument `color`.

Els colors es poden especificar amb una sola lletra per als colors bàsics: 'b' per blau, 'g' per verd, 'r' per vermell, 'c' per cian, 'm' per magenta, 'y' per groc, 'k' per negre, i 'w' per blanc.

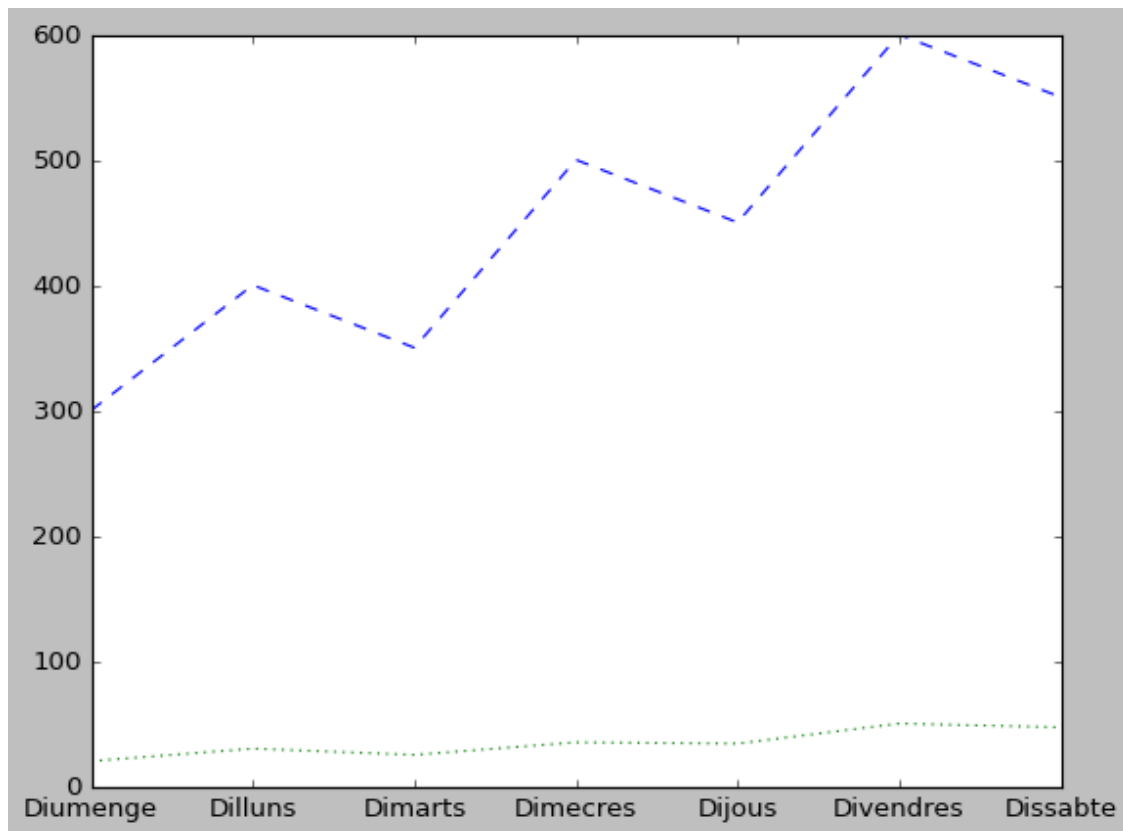
A més d'aquestes abreviatures dels noms dels colors també poden utilitzar els noms de colors complets com 'blue', 'green', 'red', etc., o codis de color hexadecimals com '#FF5733'.

```
[ ]: plt.plot(data['Dia'],data['Visites'],color='magenta')
plt.plot(data['Dia'],data['Vendes'],color='#FF007F')
plt.show()
```



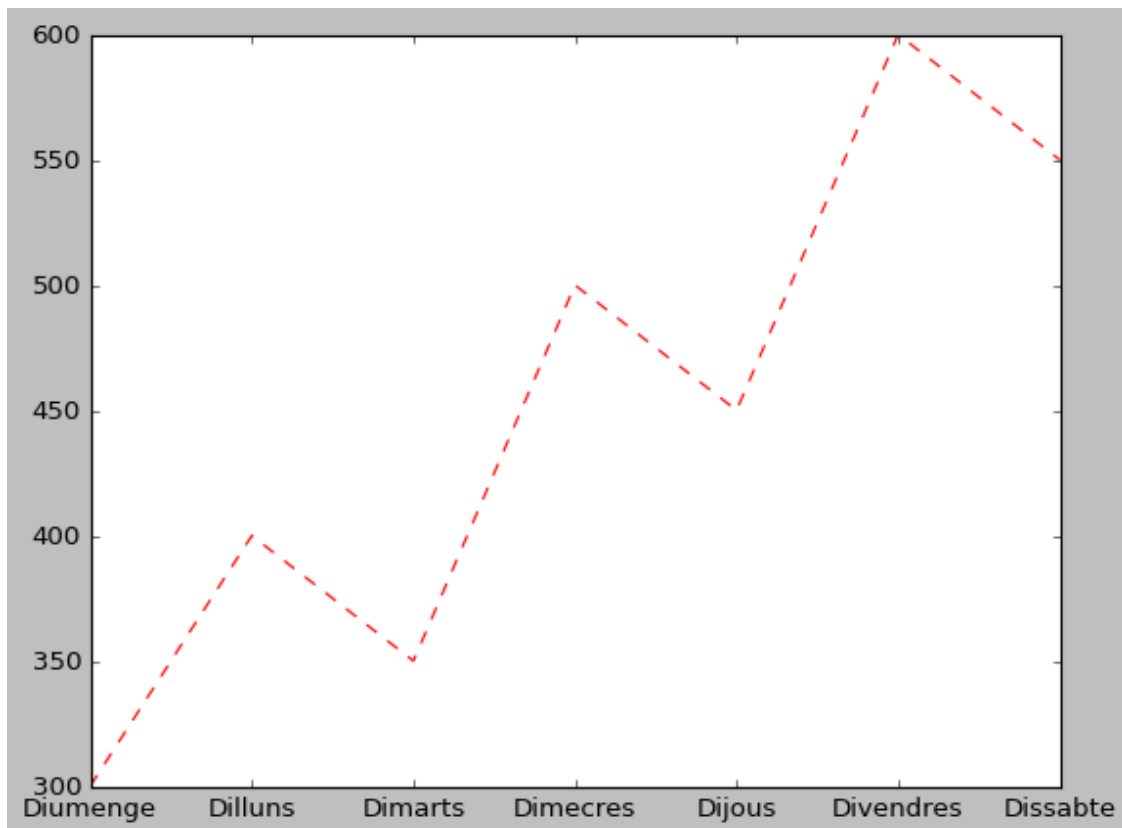
Estil de la línia Matplotlib permet personalitzar l'estil de les línies en els gràfics mitjançant a través de l'argument `linestyle`. Els estils de línia es poden especificar utilitzant cadenes de caràcters: '-' per a una línia contínua, '-' per a una línia discontinua, '-.' per a una línia amb punts i guions, i ':' per a una línia de punts.

```
[ ]: plt.plot(data['Dia'],data['Visites'],linestyle='--')  
plt.plot(data['Dia'],data['Vendes'],linestyle=':')  
plt.show()
```



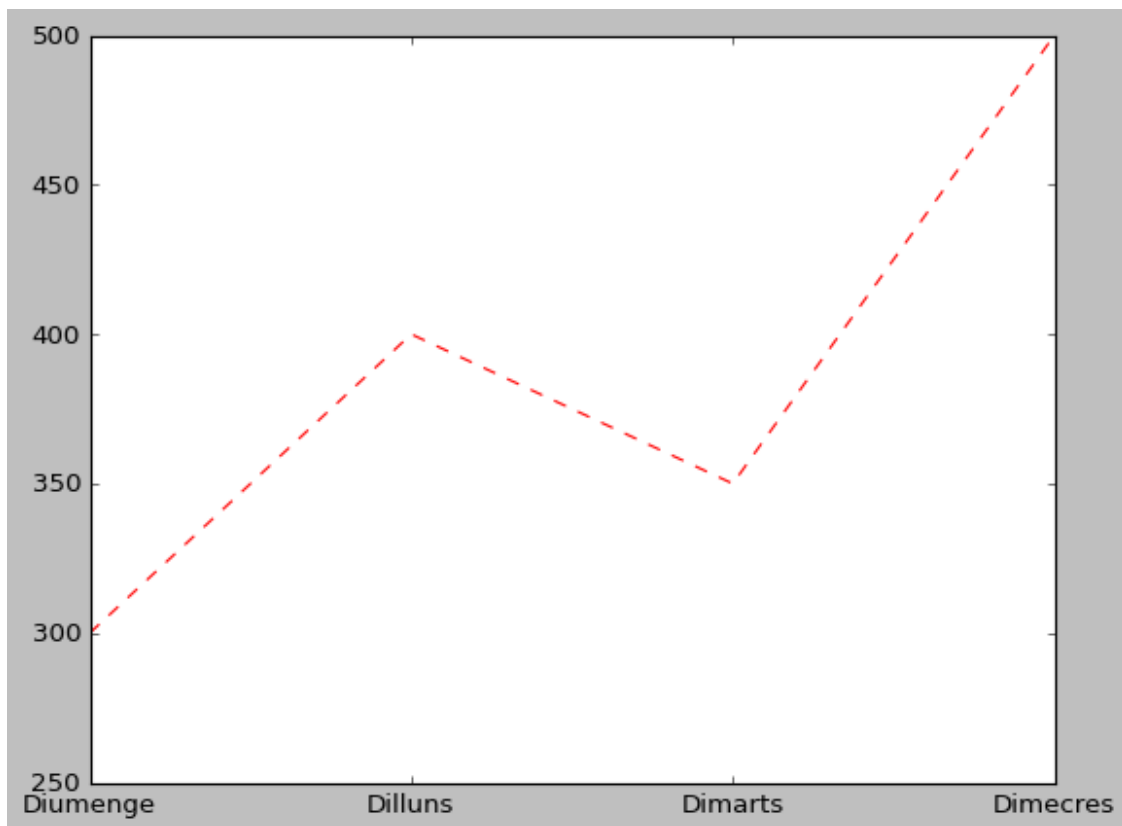
També podem especificar un estil de línia combinat amb un color com `'-r'` (una línia discontinua de color vermell) utilitzant el tercer argument de la funció `plot()`.

```
[ ]: plt.plot(data['Dia'],data['Visites'], '--r')  
plt.show()
```



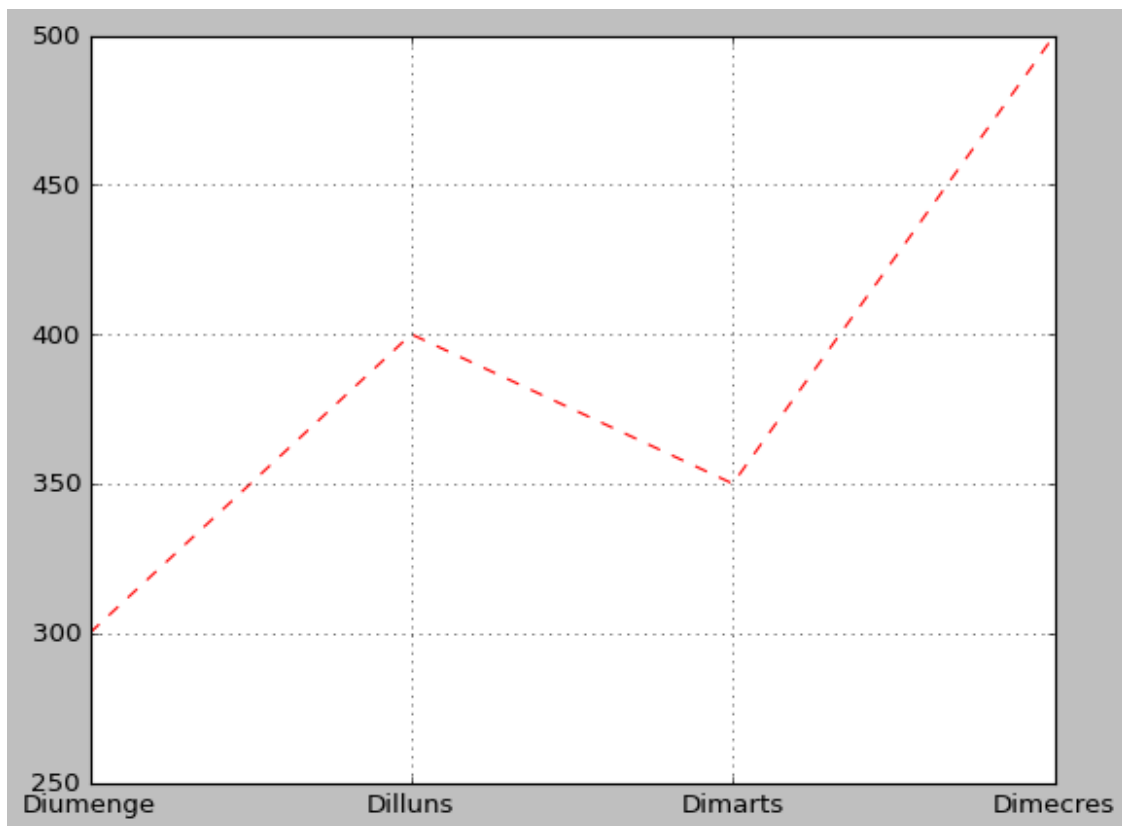
Definir el límits dels eixos Sino ho especifiquem, Matplotlib escull per nosaltres els límits dels eixos en els teus nostres gràfics intentant una representació òptima. Però de vegades ens interessa representar només una part del gràfic o que es veguin uns valors determinats i la manera de fer-ho és ajustant els límits dels eixos utilitzant els mètodes `plt.xlim()` i `plt.ylim()`.

```
[ ]: plt.plot(data['Dia'], data['Visites'], '--r')
      plt.xlim(0, 3) # Estableix el límit de l'eix x entre 0 i 3 (de dilluns a
                     ↪ dimecres)
      plt.ylim(250, 500) # Estableix el límit de l'eix y entre 250 i 500
      plt.show()
```



També podem utilitzar la funció `plt.grid()` per afegir una quadrícula al teu gràfic, que ens ajudarà a visualitzar millor l'origen i altres punts de referència.

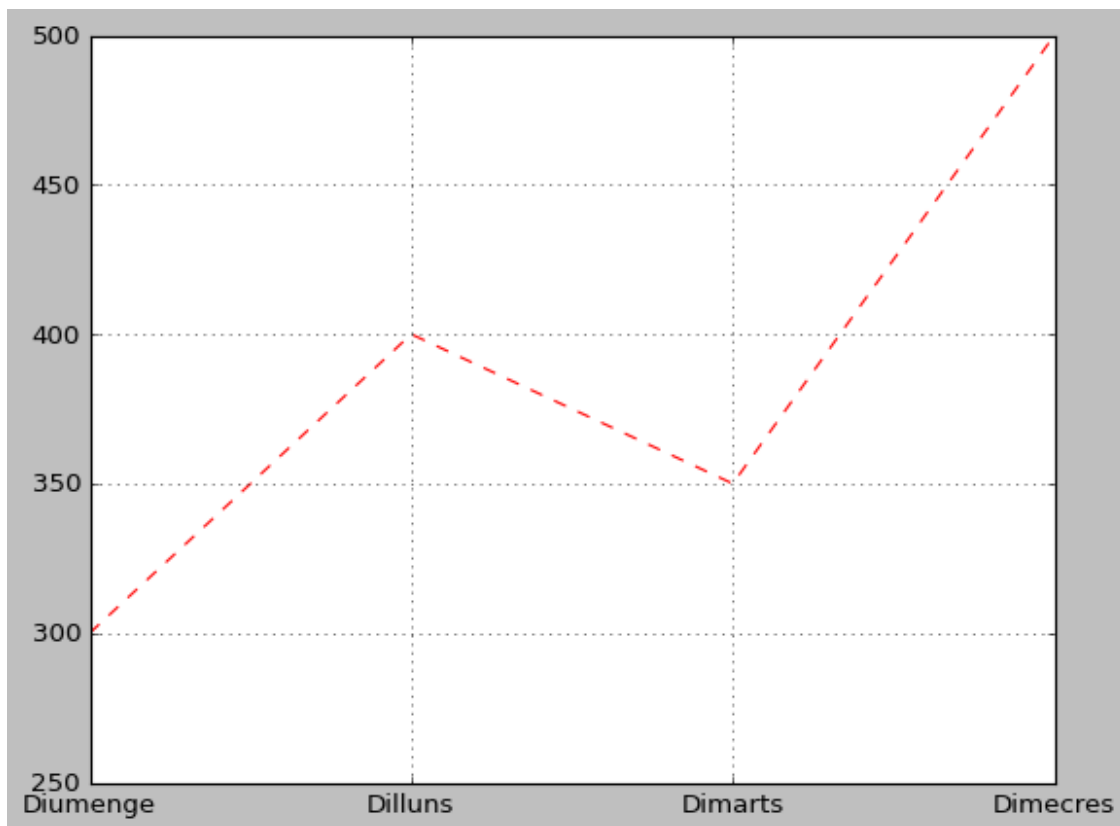
```
[ ]: plt.plot(data['Dia'], data['Visites'], '--r')  
plt.xlim(0, 3) # Estableix el límit de l'eix x entre 0 i 3 (de dilluns a  
             ↪ dimecres)  
plt.ylim(250, 500) # Estableix el límit de l'eix y entre 250 i 500  
plt.grid()  
plt.show()
```



Una altra funció relacionada amb la configuració dels límits dels eixos és `plt.axis()` (no confondre amb `axes` amb una “e”).

El mètode `plt.axis()` permet establir els límits dels eixos `x` i `y` en una sola crida, passant una llista que especifica `[xmin, xmax, ymin, ymax]`.

```
[ ]: plt.plot(data['Dia'],data['Visites'], '--r')  
      plt.axis([0, 3,250,500])  
      plt.grid()  
      plt.show()
```



Afegir text als gràfics També podem afegir text que acompanyi als gràfics per millorar-ne la lectura.

En els exemples veurem com incloure títols, etiquetar els eixos i afegir llegendes senzilles.

Per afegir un títol al gràfic que, per exemple, en descrigui el contingut, utilitzarem el mètode `plt.title`.

```
[ ]: plt.plot(data['Dia'],data['Visites'])  
plt.title("Nombre de visites segons el dia de la setmana")  
plt.show()
```



Per afegir etiquetes descriptives als eixos x i y d'un gràfic utilitzarem les funcions `plt.xlabel` i `plt.ylabel`. Aquestes etiquetes ajuden a indicar què representen les dades en cada eix, millorant la comprensió i llegibilitat del gràfic.

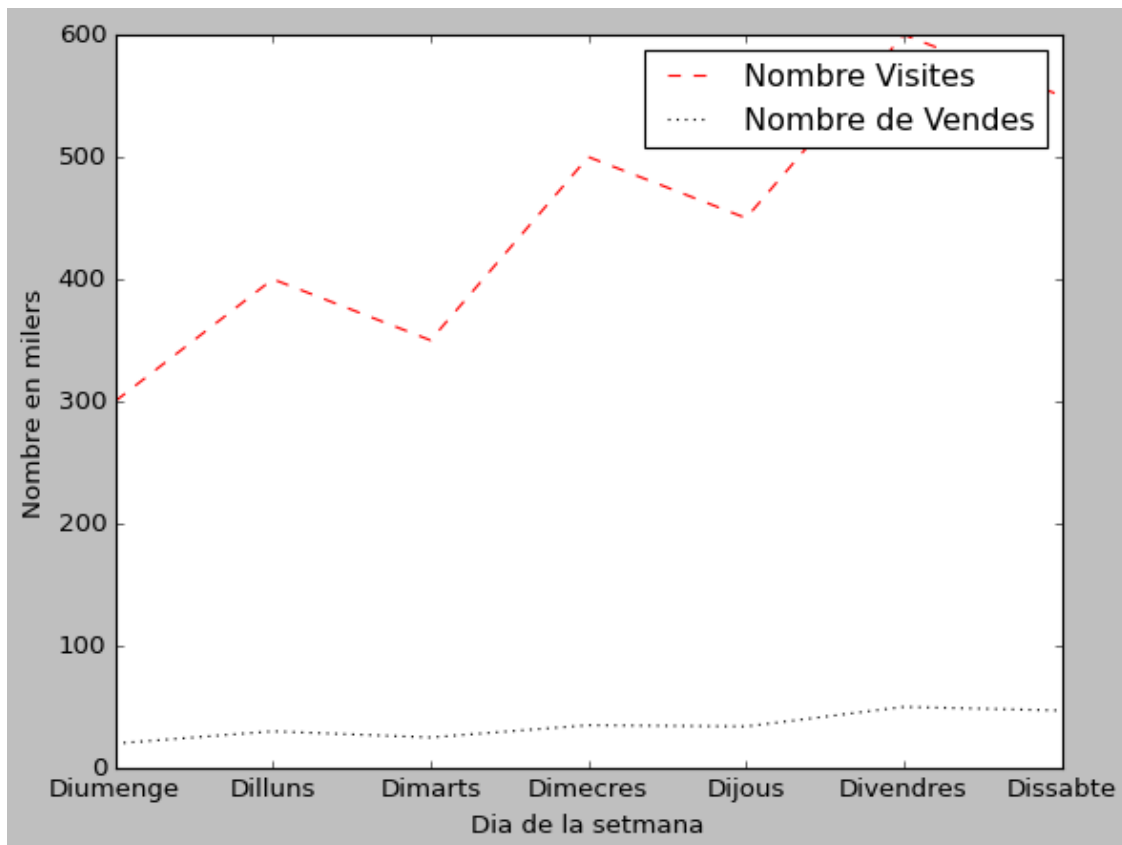
```
[ ]: plt.plot(data['Dia'],data['Visites'])  
plt.title("Nombre de visites segons el dia de la setmana")  
plt.xlabel("Dia de la setmana")  
plt.ylabel("Nombre de visites(milers)")  
plt.show()
```



Quan es mostren diverses línies o conjunts de dades dins d'una mateixa figura s'aconsella afegir una llegenda per millorar-ne la comprensió.

Podem afegir una llegenda a la figura activa a través de la funció `plt.legend()` que conté informació dels diferents elements gràfics que tenen definida la variable `label`.

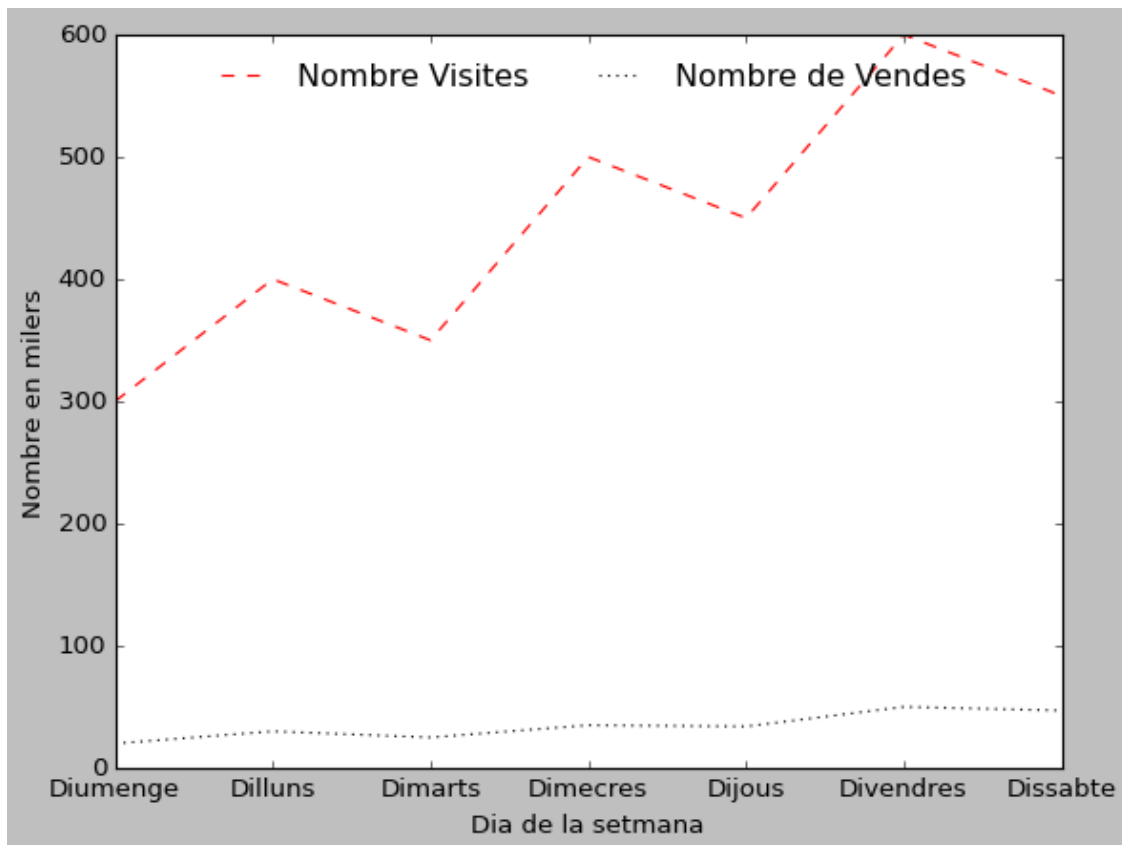
```
[ ]: plt.plot(data['Dia'],data['Visites'],'--r',label = 'Nombre Visites')
plt.plot(data['Dia'],data['Vendes'],':k', label = 'Nombre de Vendes')
plt.xlabel("Dia de la setmana")
plt.ylabel("Nombre en milers")
plt.legend()
plt.show()
```



Podem arribar a personalitzar amb més paràmetres la llegenda.

A continuació trobem un exemple:

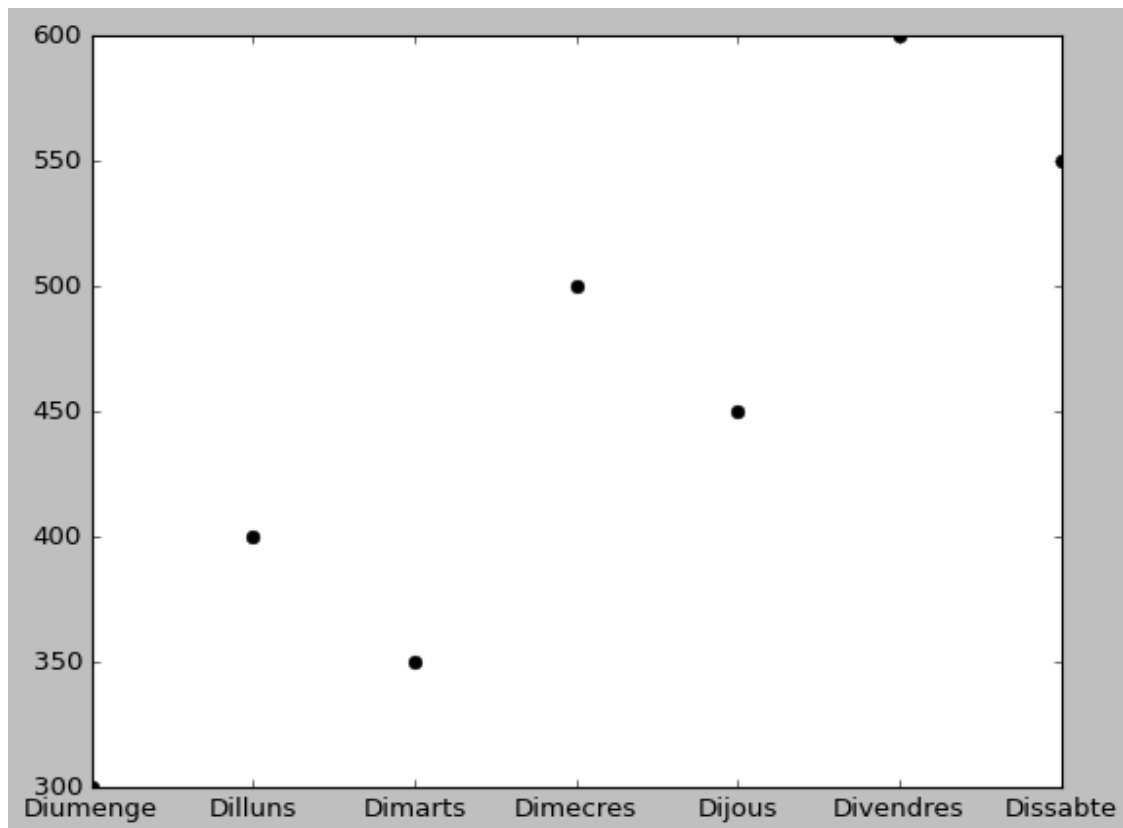
```
[ ]: plt.plot(data['Dia'],data['Visites'],'--r',label = 'Nombre Visites')
plt.plot(data['Dia'],data['Vendes'],':k', label = 'Nombre de Vendes')
plt.xlabel("Dia de la setmana")
plt.ylabel("Nombre en milers")
plt.legend(frameon=False, loc='upper center', ncol=2)
plt.show()
```



1.0.5 Gràfics de Dispersió Simples

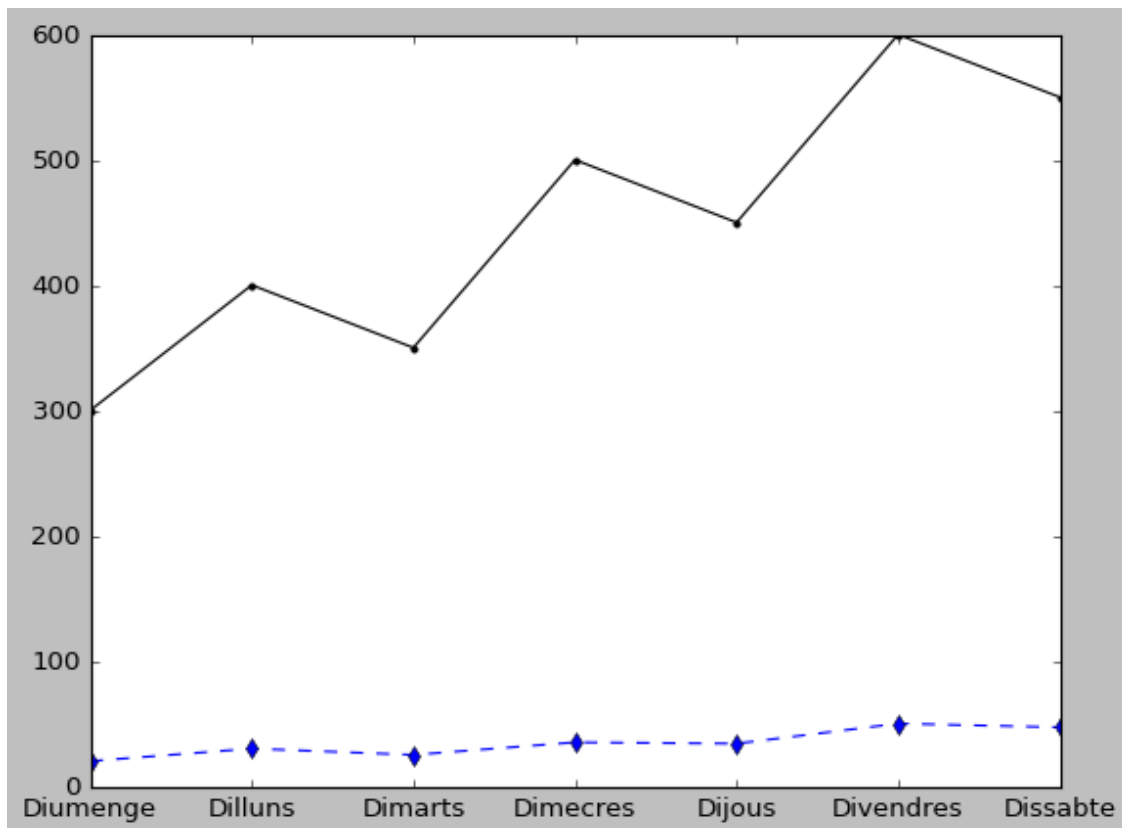
Un altre tipus de gràfic molt utilitzat és el gràfic de dispersió (*scatter plot*). És una representació similar a la lineal però en comptes que els punts estiguin units per una línia els punts es representen individualment amb un punt, cercle o una altra forma (*marker*).

```
[ ]: plt.plot(data['Dia'],data['Visites'],'o',color='black')  
plt.show()
```



El tercer argument en la crida de la funció és un caràcter que representa el tipus de símbol utilitzat com a *marker* (algunes opcions són: 'o', ',', 'x', '+', 'v', '^', '<', '>', 's', 'd'). Però també es poden especificar opcions com '-' o '-' per controlar l'estil de la línia.

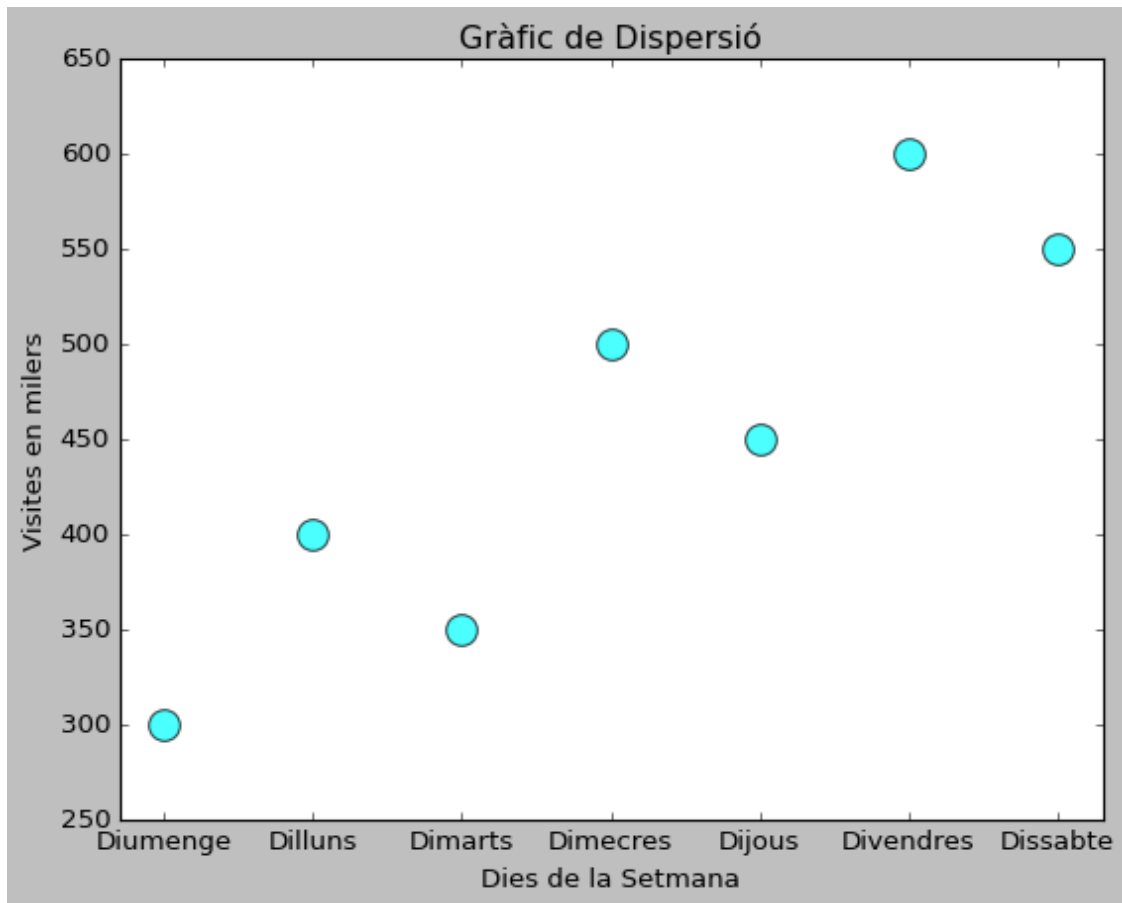
```
[ ]: plt.plot(data['Dia'],data['Visites'],'-k.')  
plt.plot(data['Dia'],data['Vendes'],'--bd')  
plt.show()
```



Un mètode més potent per crear gràfics de dispersió és la funció `plt.scatter`.

Aquest mètode ofereix més opcions de personalització, ja que permet especificar no només les coordenades *x* i *y* dels punts, sinó també altres atributs com la mida, el color i els tipus de marcadors.

```
[ ]: plt.scatter(data['Dia'],data['Visites'], s=200, c='cyan', edgecolors='black',
    ↪alpha=0.7, marker='o')
plt.title('Gràfic de Dispersió')
plt.xlabel('Dies de la Setmana')
plt.ylabel('Visites en milers')
plt.show()
```



La principal diferència entre `plt.scatter` i `plt.plot` és que `plt.scatter` es pot utilitzar per crear gràfics de dispersió on les propietats de cada punt (*marker*) es poden personalitzar de forma individual (mida, color de contorn, color de fons, etc.).

A continuació es mostra un exemple avançat perquè en pogueu veure les possibilitats però la realització d'aquest tipus de gràfics no està dintre dels objectius del curs.

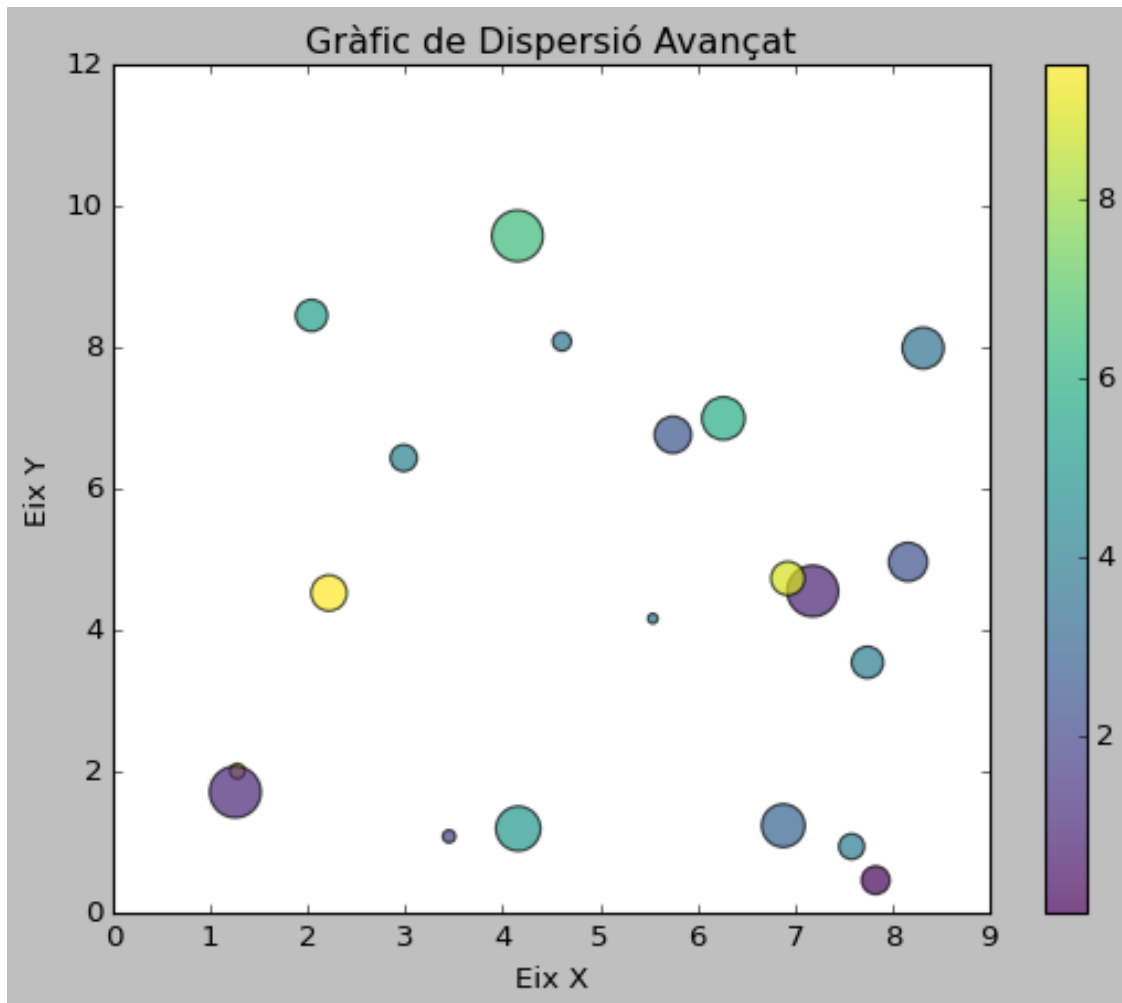
```
[ ]: import matplotlib.pyplot as plt
import numpy as np

x = np.random.rand(20) * 10 # 20 punts aleatoris en l'interval [0, 10]
y = np.random.rand(20) * 10 # 20 punts aleatoris en l'interval [0, 10]
sizes = np.random.rand(20) * 500 # Mides aleatòries per als punts
colors = np.random.rand(20) * 10 # Colors aleatoris per als punts

plt.scatter(x, y, s=sizes, c=colors, cmap='viridis', edgecolors='black',
            alpha=0.7, marker='o')

plt.title('Gràfic de Dispersió Avançat')
plt.xlabel('Eix X')
```

```
plt.ylabel('Eix Y')
plt.colorbar() # Afegir una barra de color per mostrar la escala de colors
plt.show()
```



Ús d'objectes tipus Figure i Axes A l'hora de representar gràfics amb Matplotlib, tenim dues maneres de construir-los: utilitzant únicament les funcions de plt o utilitzant un enfocament orientat a objectes a través d'utilitzar els objectes Figure i Axes.

L'objecte Figure fa referència a la finestra o llenç i permet, per exemple, la creació de múltiples gràfics dins d'una sola finestra.

L'objecte Axes fa referència a l'àrea dins de la figura on es dibuixa el gràfic i ofereix diverses opcions per personalitzar-lo.

Com ja hem comentat, hi ha dues maneres d'utilitzar Matplotlib per crear gràfics. La primera és la més senzilla, basada en la interfície funcional amb el mòdul pyplot (plt), on crides directament funcions com plot(), bar() o scatter() i Matplotlib crea i gestiona automàticament la Figure i els

Axes. Aquesta manera és molt còmoda per a gràfics simples o prototips ràpids.

La segona manera és l'enfocament orientat a objectes, on es crea explícitament una Figure i un o diversos Axes. Això et dóna molt més control sobre cada element gràfic. Per exemple, amb `fig, ax = plt.subplots()` s'obtenen un objecte Figure i un Axes que després pots manipular directament amb mètodes com `ax.plot()`, `ax.set_title()`, o `ax.legend()`. Aquesta manera és la recomanada per projectes més complexos o quan es vol fer una personalització fina.

Nosaltres ens centrarem en les funcions de `plt` per la seva simplicitat, però és bo que conegueu també la manera de treballar amb axes i figure per si en algun moment consulteu documentació o exemples. A més, tret d'alguns casos més complexos, es pot “traduir” d'un mètode a l'altre amb resultats pràcticament idèntics.

A continuació observem dos exemples amb resultat similar: un realitzat amb funcions de `plt` i l'altre fent ús de Figure i Axes.

Exemples basats en part del codi de l'article [“Matplotlib line plots- when and how to use them”](#)

```
[ ]: plt.style.use('seaborn-v0_8-talk')
```

```
[ ]: import matplotlib.pyplot as plt
```

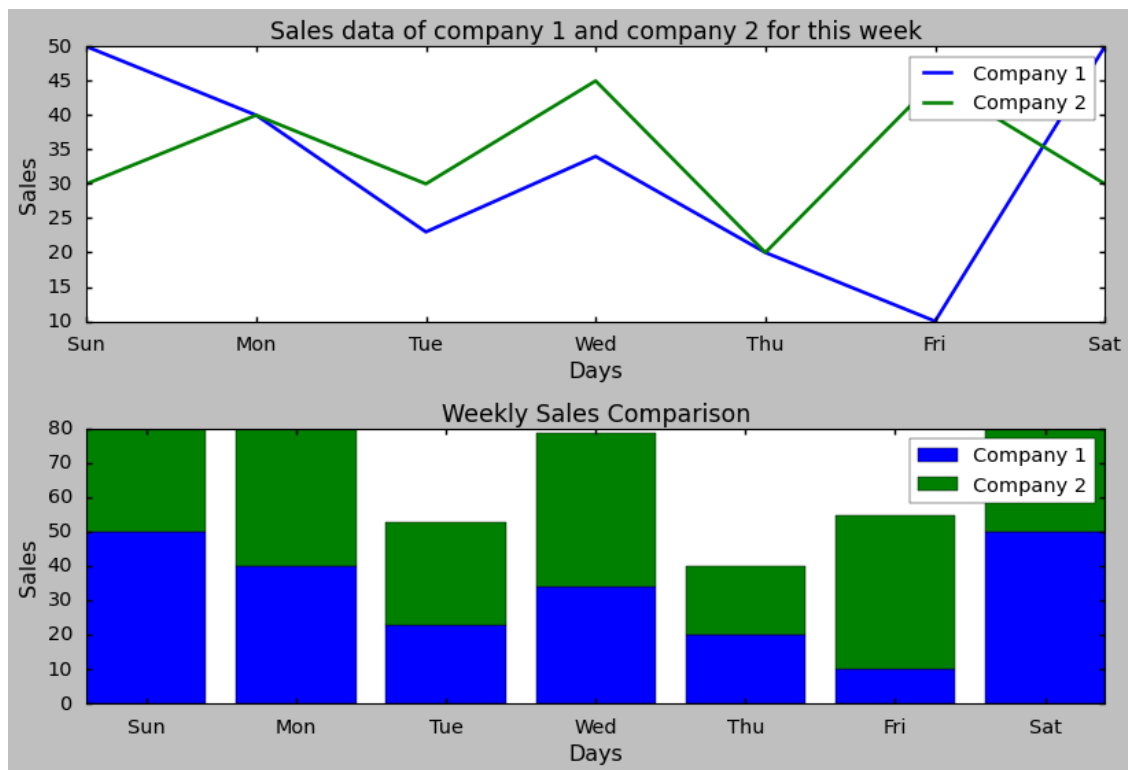
```
days = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]
company1 = [50, 40, 23, 34, 20, 10, 50] # sales of company1
company2 = [30, 40, 30, 45, 20, 45, 30] # sales of company 2
```

```
[ ]: plt.figure()
```

```
plt.subplot(2, 1, 1) # Gràfic 1 a la posició 1
plt.plot(days, company1, label = "Company 1") # line chart company 1
plt.plot(days, company2, label = "Company 2")
plt.xlabel("Days")
plt.ylabel("Sales")
plt.legend() # using legend to differentiate the graphs of companies
plt.title("Sales data of company 1 and company 2 for this week")
```

```
plt.subplot(2, 1, 2)
plt.bar(days, company1, label='Company 1')
plt.bar(days, company2, bottom=company1, label='Company 2')
plt.xlabel('Days')
plt.ylabel('Sales')
plt.title('Weekly Sales Comparison')
plt.legend()

plt.tight_layout() #ajust automàtic de la distància entre gràfics
plt.show()
```

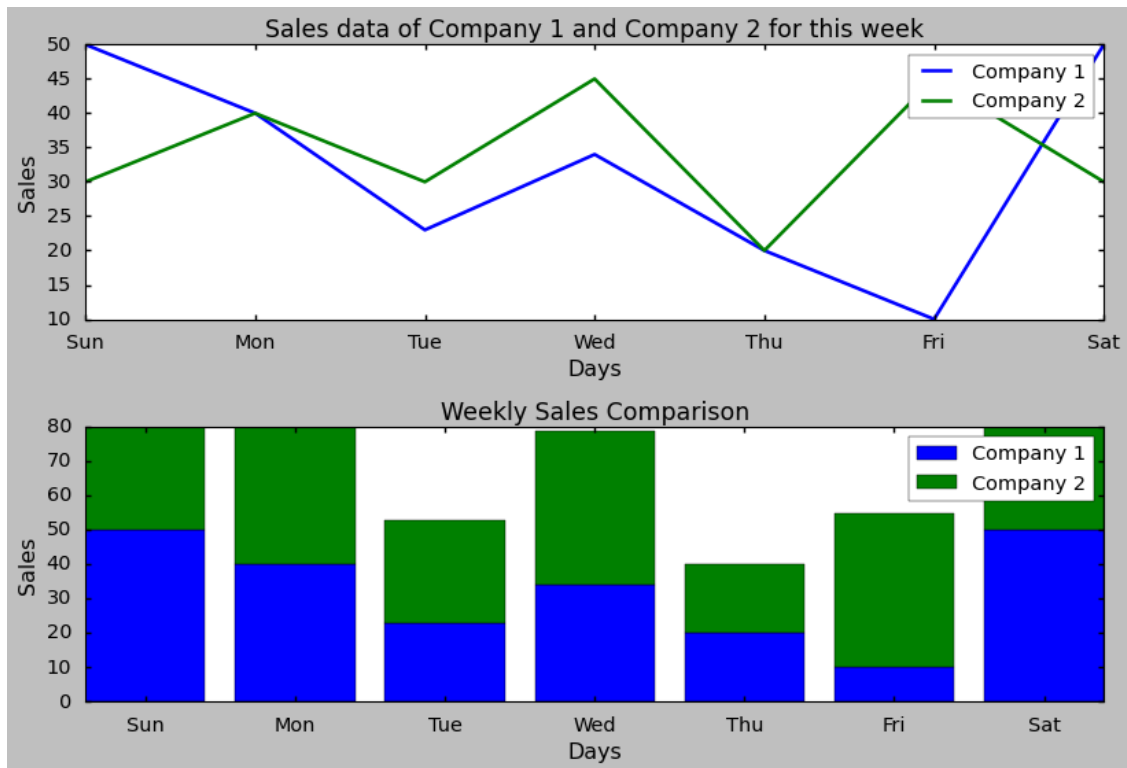


```
[ ]: fig, axs = plt.subplots(2, 1) # 2 files, 1 columna

axs[0].plot(days, company1, label="Company 1")
axs[0].plot(days, company2, label="Company 2")
axs[0].set_xlabel("Days")
axs[0].set_ylabel("Sales")
axs[0].set_title("Sales data of Company 1 and Company 2 for this week")
axs[0].legend()

axs[1].bar(days, company1, label='Company 1')
axs[1].bar(days, company2, bottom=company1, label='Company 2')
axs[1].set_xlabel('Days')
axs[1].set_ylabel('Sales')
axs[1].set_title('Weekly Sales Comparison')
axs[1].legend()

plt.tight_layout() #ajust automàtic de la distància entre gràfics
plt.show()
```



1.0.6 Seaborn versus Matplotlib

Matplotlib la llibreria de creació de gràfics típica de Python. Permet crear gràfics de tot tipus i permet a l'usuari configurar-ne qualsevol aspecte: colors, etiquetes, llegendes, etc.

Seaborn és una llibreria construïda sobre Matplotlib que afegeix una capa de creació de gràfics estèticament agradables i moderns. Es dissenya especialment per a treballar amb *DataFrames* de pandas, cosa que la fa ideal per a l'anàlisi de dades. Ofereix gràfics amb un estil més modern. Inclou paletes de colors i gràfics predefinits que són atractius visualment sense necessitat de molta personalització.

Podríem dir que Seaborn és una versió millorada i una eina molt intuïtiva si necessites visualitzar dades emmagatzemades en forma de *DataFrame*.

Veiem a continuació dos exemples de les mateixes dades: un fent ús de matplotlib i l'altre fent ús de matplotlib però amb els extrems que ens proporciona Seaborn.

```
[ ]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

df = pd.DataFrame({'Any': [2000, 2005, 2010, 2015, 2020],
                  'Nombre_Habitants': [150000, 180000, 200000, 220000, 250000]})
```

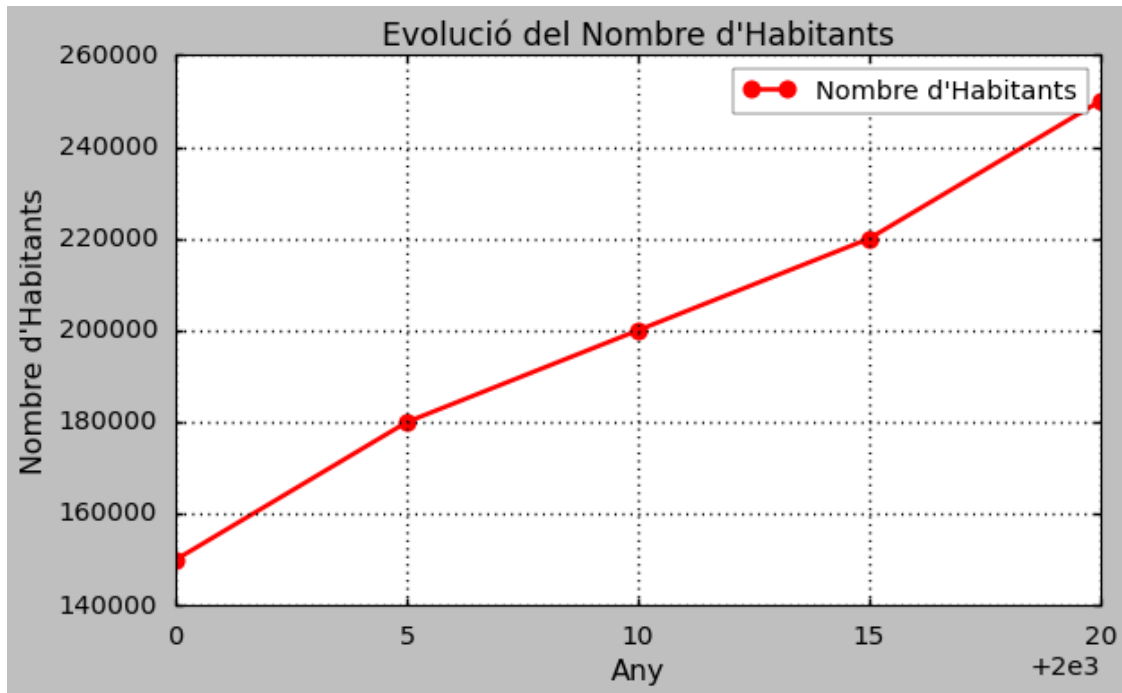
Representació només amb Matplotlib:

```
[ ]: plt.figure(figsize=(8,5)) #opcionalment en definim la mida.

#MATPLOTLIB
plt.plot(df['Any'], df['Nombre_Habitants'], marker='o', color='red',
        label='Nombre d\'Habitants')

plt.title('Evolució del Nombre d\'Habitants')
plt.xlabel('Any')
plt.ylabel('Nombre d\'Habitants')
plt.xticks(df['Any'])
plt.grid(True)
plt.legend()
plt.tight_layout() #Evita que els elements es solapin

plt.show()
```



Exemple realitzat amb Matplotlib + Seaborn:

```
[ ]: plt.figure(figsize=(8, 5)) #opcionalment en definim la mida.

#SEABORN: ens permet ajustar l'estil i el context
sns.set_style("darkgrid") # Estil de fons amb quadrícula blanca
sns.set_context("talk") # Context ajustat per a presentacions
```

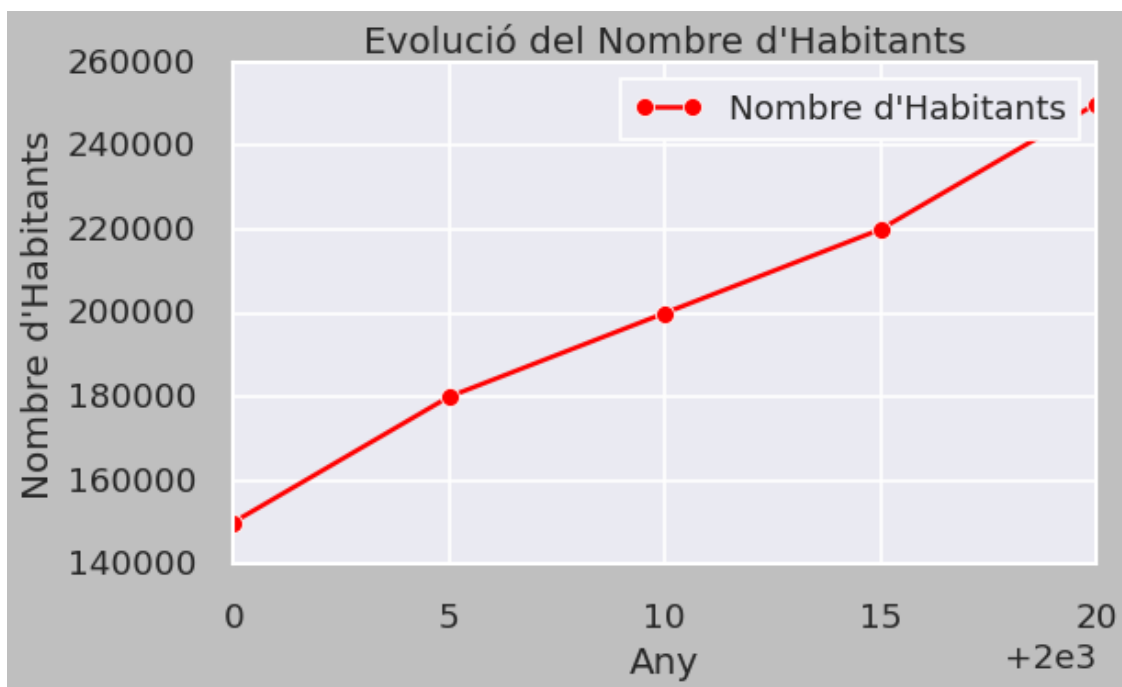
```

sns.lineplot(x='Any', y='Nombre_Habitants', data=df, marker='o',
             color='red', label='Nombre d\'Habitants')

plt.title('Evolució del Nombre d\'Habitants')
plt.xlabel('Any')
plt.ylabel('Nombre d\'Habitants')
plt.xticks(df['Any'])
plt.grid(True)
plt.legend()
plt.tight_layout() #Evita que els elements es solapin

plt.show()

```



1.1 Exemples de representacions de Dataframes amb Seaborn

1.1.1 Exemple: Característiques Morfològiques de Pingüins (Scatter Plot)

Dataset: penguins El dataset “penguins” és una col·lecció de dades sobre diferents espècies de pingüins recollides per científics, que inclou mesures físiques com la longitud de la aleta (flipper length), la longitud del bec (bill length), i el pes corporal, així com la classificació de l'espècie

```

[ ]: # Carregar dataset "penguins"
df_penguins = sns.load_dataset("penguins")

df_penguins

```

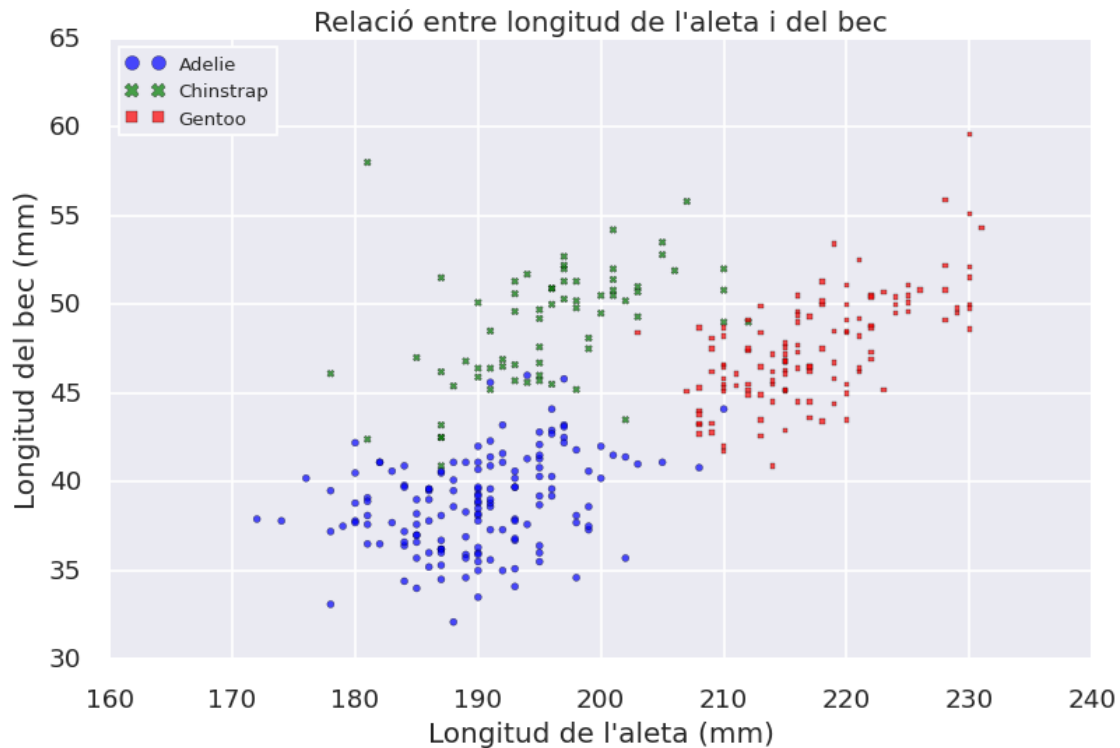
```
[ ]: species      island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0    Adelie  Torgersen      39.1          18.7           181.0
1    Adelie  Torgersen      39.5          17.4           186.0
2    Adelie  Torgersen      40.3          18.0           195.0
3    Adelie  Torgersen      NaN          NaN            NaN
4    Adelie  Torgersen      36.7          19.3           193.0
..      ...      ...
339  Gentoo  Biscoe        NaN          NaN            NaN
340  Gentoo  Biscoe        46.8          14.3           215.0
341  Gentoo  Biscoe        50.4          15.7           222.0
342  Gentoo  Biscoe        45.2          14.8           212.0
343  Gentoo  Biscoe        49.9          16.1           213.0

      body_mass_g      sex
0          3750.0    Male
1          3800.0  Female
2          3250.0  Female
3           NaN     NaN
4          3450.0  Female
..      ...      ...
339         NaN     NaN
340         4850.0  Female
341         5750.0    Male
342         5200.0  Female
343         5400.0    Male

[344 rows x 7 columns]
```

```
[ ]: # Gràfic de dispersió
sns.scatterplot(
    data=df_penguins,
    x="flipper_length_mm",
    y="bill_length_mm",
    hue="species",           # Color per espècie
    style="species",         # Forma del marcador per espècie
    sizes=(20, 200),        # Rang de mides
    alpha=0.7,              # Transparència
    edgecolor="black"       # Contorn negre per millor visibilitat
)

plt.title('Relació entre longitud de l\'aleta i del bec')
plt.xlabel('Longitud de l\'aleta (mm)')
plt.ylabel('Longitud del bec (mm)')
plt.legend(loc='best',fontsize=12)
plt.tight_layout()
plt.show()
```



1.1.2 Exemple: Propines en Restaurants (Barplot)

Dataset: tips El dataset “tips” és un conjunt de dades recopilades a un restaurant on es registra la informació de les propines (“tip”), el compte total del client (“total_bill”), el dia de la setmana, el moment del dia (dinars o sopars), i altres variables relacionades amb els clients i les seves comptes.

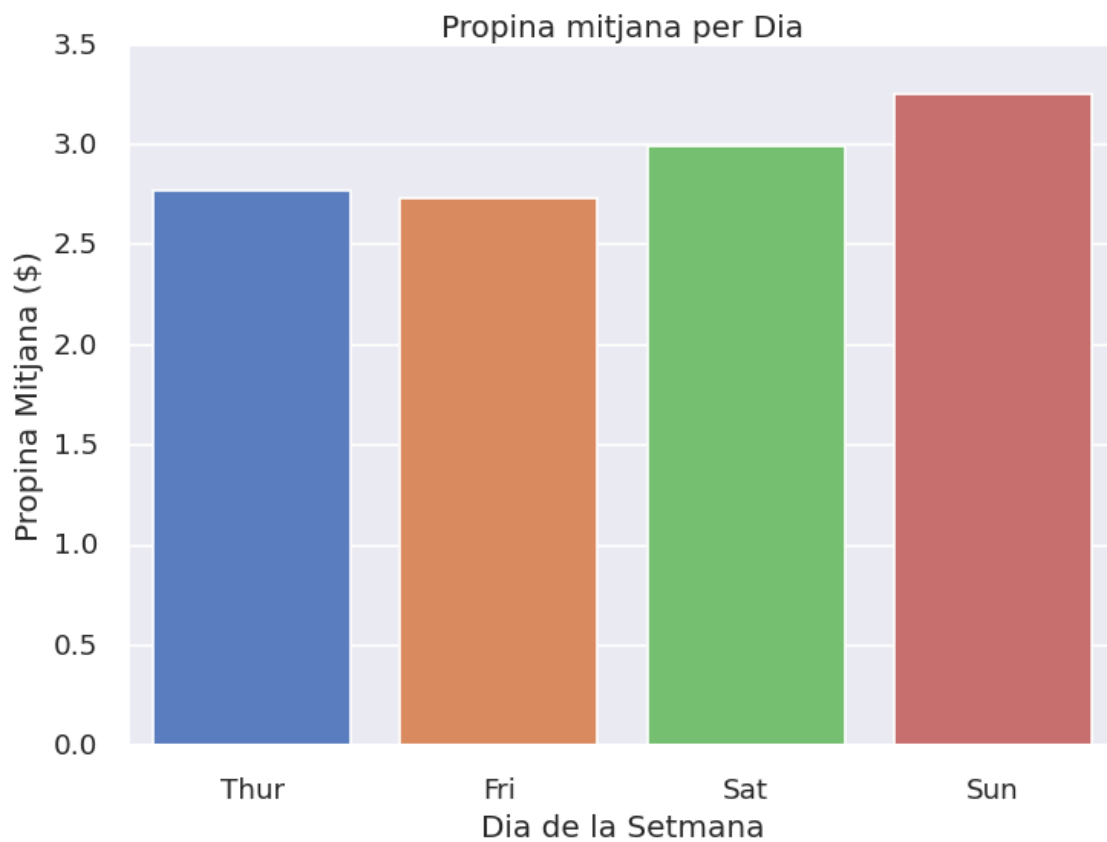
```
[ ]: # Carregar dades "tips"
tips = sns.load_dataset("tips")
tips
```

```
[ ]:      total_bill  tip  sex smoker  day  time  size
0         16.99  1.01 Female    No  Sun  Dinner     2
1         10.34  1.66  Male    No  Sun  Dinner     3
2         21.01  3.50  Male    No  Sun  Dinner     3
3         23.68  3.31  Male    No  Sun  Dinner     2
4         24.59  3.61 Female    No  Sun  Dinner     4
..         ...   ...   ...    ...  ...   ...   ...
239        29.03  5.92  Male    No  Sat  Dinner     3
240        27.18  2.00 Female   Yes  Sat  Dinner     2
241        22.67  2.00  Male   Yes  Sat  Dinner     2
242        17.82  1.75  Male    No  Sat  Dinner     2
243        18.78  3.00 Female    No  Thur Dinner     2
```

[244 rows x 7 columns]

```
[ ]: # Crear barplot
sns.barplot(
    data=tips,
    x="day",
    y="tip",
    hue="day",
    errorbar=None,
    palette="muted",
    legend=False      # No volem llegenda
)

plt.title('Propina mitjana per Dia')
plt.xlabel('Dia de la Setmana')
plt.ylabel('Propina Mitjana ($)')
plt.show()
```



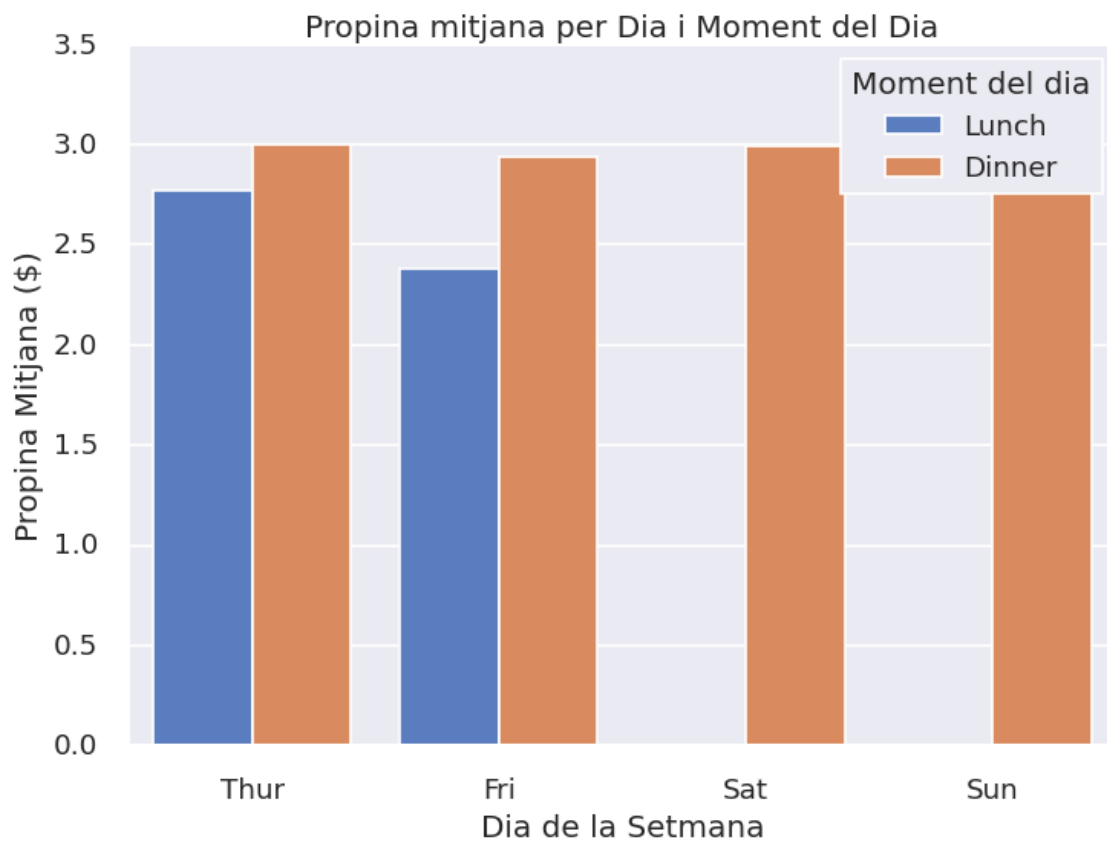
```
[ ]: sns.barplot(
    data=tips,
```

```

x="day",
y="tip",
hue="time",          # Diferenciar per moment del dia dins cada dia
errorbar=None,
palette="muted"
)

plt.title('Propina mitjana per Dia i Moment del Dia')
plt.xlabel('Dia de la Setmana')
plt.ylabel('Propina Mitjana ($)')
plt.legend(title='Moment del dia')
plt.show()

```



1.1.3 Exemple: Passatgers Aeris Mensuals (Histograma i gràfic lineal)

Dataset: flights Informació històrica sobre el nombre de passatgers aèreos mensuals en vols comercials de la companyia americana Pan Am durant els anys 1949 a 1960.

```

[ ]: import seaborn as sns
import matplotlib.pyplot as plt

```

```
# Carregar dataset flights
flights = sns.load_dataset("flights")
flights
```

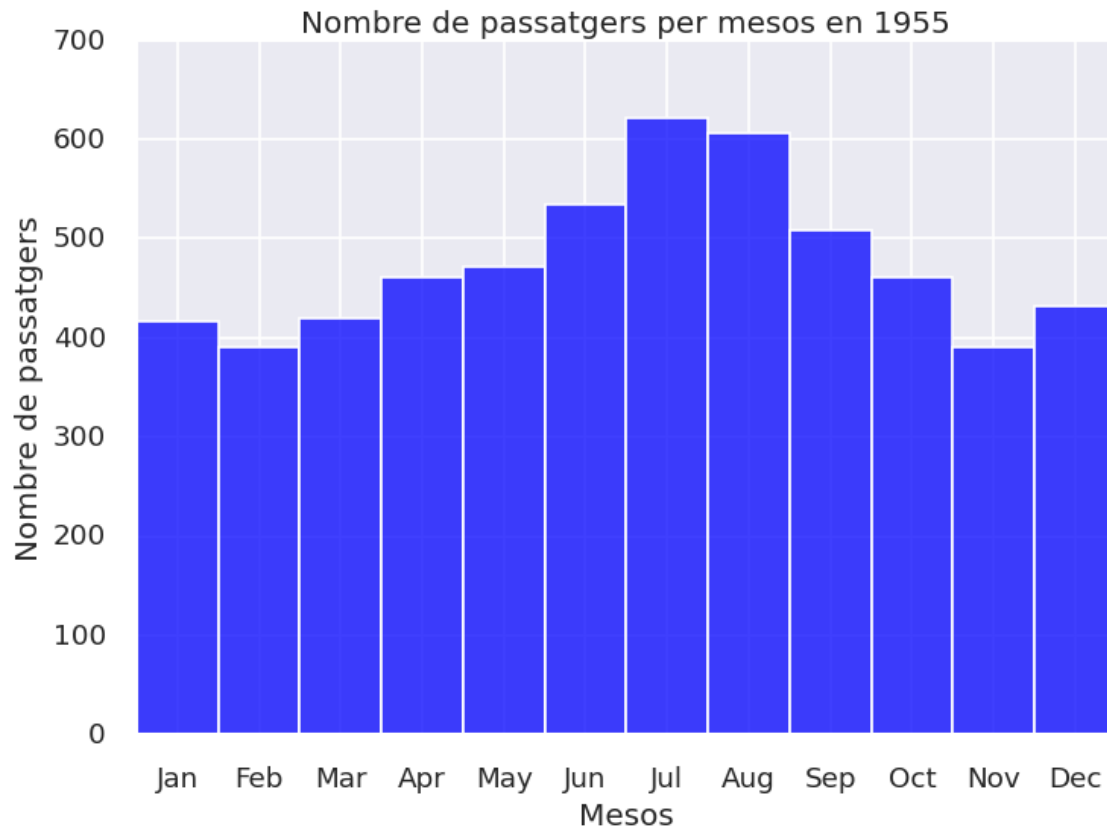
```
[ ]:      year month  passengers
0    1949   Jan         112
1    1949   Feb         118
2    1949   Mar         132
3    1949   Apr         129
4    1949   May         121
..    ...   ...         ...
139  1960   Aug         606
140  1960   Sep         508
141  1960   Oct         461
142  1960   Nov         390
143  1960   Dec         432
```

[144 rows x 3 columns]

```
[ ]: flights_1960 = flights[flights['year'] == 1960]

# Histograma del nombre de passatgers per mesos (x = mes, y = freq)
sns.histplot(
    data=flights_1960,
    x="month",
    weights="passengers", # Peseu les barres per nombre de passatgers
    discrete=True,
    color='blue'
)

plt.title("Nombre de passatgers per mesos en 1955")
plt.xlabel("Mesos")
plt.ylabel("Nombre de passatgers")
plt.show()
```



```
[ ]: # Agrupar dades per any per veure la tendència anual
yearly = flights.groupby('year')['passengers'].sum()
yearly
```

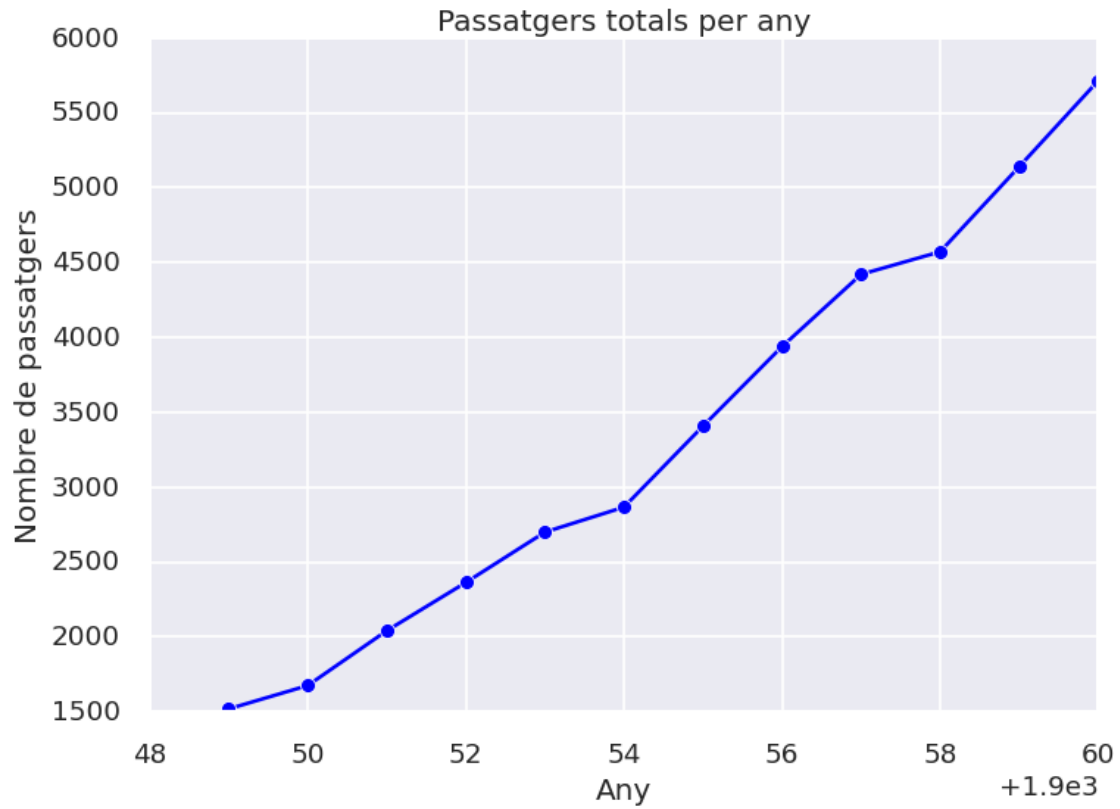
```
[ ]: year
1949    1520
1950    1676
1951    2042
1952    2364
1953    2700
1954    2867
1955    3408
1956    3939
1957    4421
1958    4572
1959    5140
1960    5714
Name: passengers, dtype: int64
```

```
[ ]: # Agrupar dades per any per veure la tendència anual
yearly = flights.groupby('year')['passengers'].sum().reset_index()
yearly
```

```
[ ]:   year  passengers
0   1949         1520
1   1950         1676
2   1951         2042
3   1952         2364
4   1953         2700
5   1954         2867
6   1955         3408
7   1956         3939
8   1957         4421
9   1958         4572
10  1959         5140
11  1960         5714
```

```
[ ]: # Gràfic de línies de passatgers totals per any
sns.lineplot(data=yearly, x='year', y='passengers', marker='o')

plt.title('Passatgers totals per any')
plt.xlabel('Any')
plt.ylabel('Nombre de passatgers')
plt.grid(True)
plt.show()
```



1.1.4 Per saber-ne més:

Alguns links recomenats de consulta:

- <https://matplotlib.org/stable/gallery/index.html>
- <https://seaborn.pydata.org/examples/index.html>
- <https://python-graph-gallery.com/>